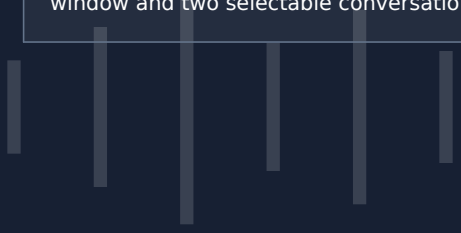
THE IDEA ARCHITECT

Standalone conversational design library

A provider-neutral Python package and CLI for interruption-friendly brainstorming, Opus peer review, and repo-native planning artifacts

FIRST TARGET	LATER DESTINATION
A personal macOS terminal companion, launched beside Claude Code, with a tiny app-mode audio window and two selectable conversation providers.	A reusable integration type, a typed agent, or both inside Dee's broader event-driven framework - without changing the standalone contracts.



ARCHITECTURAL DECISION

Prototype the behavior as its own library. Treat Parlor and Nova 2 Sonic as providers, Opus as a separate architect, and repository changes as reviewed artifacts - not side effects of the voice runtime.

Executive decision

A narrow prototype boundary with a clean path into the larger ecosystem

Build a library first; integrate it later.

The prototype should answer one hard product question: can a long, interruptible technical conversation produce trustworthy implementation planning without disrupting the normal Claude Code workflow? A standalone package makes that measurable while keeping framework-specific abstractions out of the audio experiment.

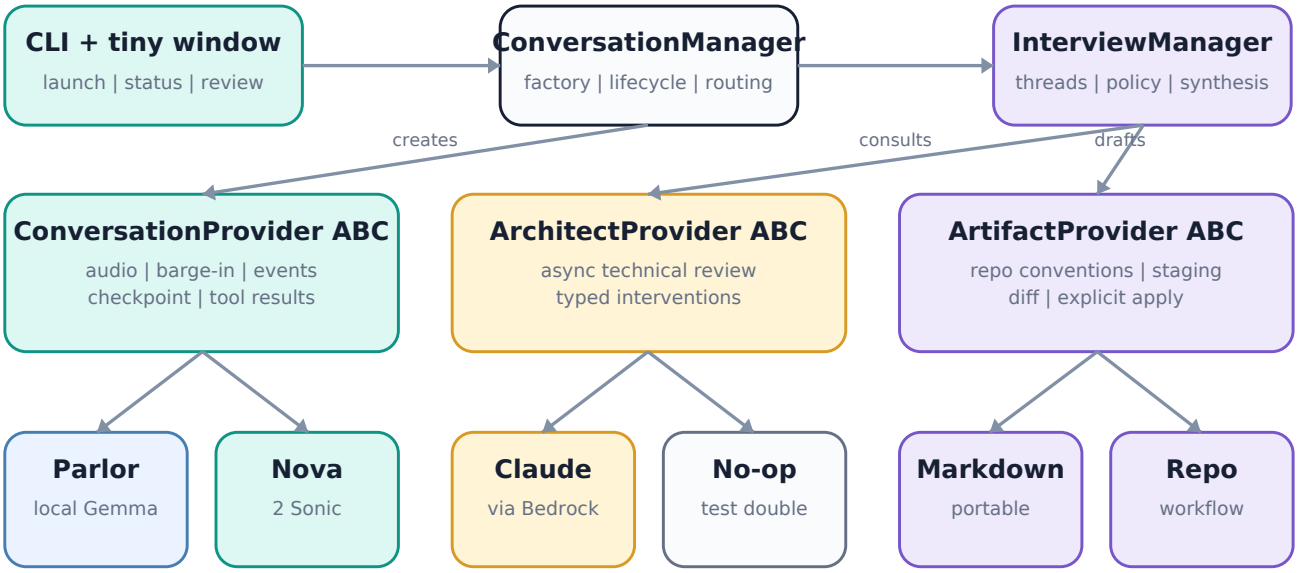
- Use a ConversationManager with factory methods to select provider implementations from configuration.
- Normalize conversation events at the library boundary; never expose Parlor, Nova, browser, or Strands event types to consumers.
- Keep the interviewer and Opus peer-review behavior separate from duplex audio mechanics.
- Inspect repositories read-only and stage proposed Markdown patches; require explicit approval before applying any change.
- Defer Claude Code invocation, Protocol Manager effects, team workflows, and AgentCore deployment until the standalone behavior is proven.

Success means

Proof	Release evidence
Natural	Three 60-minute sessions with reliable turn completion, low-friction barge-in, and usable device recovery.
Provider-neutral	The same CLI and session model run against both Parlor/Gemma and Nova 2 Sonic.
Architecturally useful	Opus can intervene asynchronously without freezing or hijacking the conversation.
Durable	A reviewed session output cleanly separates decisions, constraints, risks, open questions, tasks, and acceptance criteria.
Integration-ready	A consumer can subscribe to typed events and control lifecycle without importing provider-specific code.

System boundary and architecture

Three managers; three provider seams; one normalized event stream



Responsibility split

Component	Owns	Must not own
ConversationManager	Provider creation, lifecycle, capability negotiation, routing, checkpoint coordination	Interview content or repo policy
InterviewManager	Thread state, prompt posture, consultation triggers, confirmation, synthesis	Audio devices or model transport
ArtifactManager	Convention discovery, staging, diff, apply approval, export	Conversational timing or autonomous implementation
CLI	Configuration, launch, status, commands, review surface	Domain behavior hidden in command handlers

The event stream is the durable seam.

Managers coordinate through library-owned Pydantic event types. Providers may retain native payloads as provenance metadata, but consumers reason over stable event names, sequence numbers, timestamps, correlation IDs, and typed payloads.

Package and dependency design

Keep the public API small and the optional dependencies isolated

Package	Purpose	Dependency rule
idea_architect.conversation	Manager, ABC, capabilities, session handle, checkpoints	Core only; no Parlor, AWS, browser, or Strands imports
idea_architect.events	Discriminated event payloads and envelopes	Core only; serialization must be deterministic
idea_architect.interview	Policy, thread state, confirmation, consultation coordination	May depend on conversation/events, never on concrete providers
idea_architect.architect	Architect ABC and intervention schemas	Concrete Bedrock dependency lives in providers
idea_architect.artifacts	Repo discovery, staging, patch generation, approval	Filesystem and git adapters behind narrow ports
idea_architect.providers.parlor	Local Gemma/Parlor conversation adapter	Optional extra: parlor
idea_architect.providers.nova	Bedrock Nova 2 Sonic adapter	Optional extra: aws; Strands adapter may be separate
idea_architect.providers.claude	Bedrock Claude architect adapter	Optional extra: aws
idea_architect.cli	Typer/Rich commands and terminal review	May compose managers; contains no core rules
web/compact	Tiny microphone/playback/status UI	Provider-neutral local WebSocket contract

Suggested extras

```
pip install idea-architect[parlor]
pip install idea-architect[aws]
pip install idea-architect[all]

# Core tests must pass with no optional extra installed.
```

Conversation provider contract

Normalize observable behavior without pretending the providers are internally equivalent

Method	Contract intent
capabilities()	Declare native audio, VAD, barge-in, async tools, renewal, local execution, text injection, transcripts, and checkpoint fidelity.
start_session(context)	Open resources and return a session handle without blocking the event consumer.
send_audio(chunk)	Accept timestamped PCM frames or a negotiated audio envelope.
events()	Yield ordered library-owned events until session closure.
interrupt(reason)	Cancel current response/playback and emit a correlated interruption event.
submit_tool_result(result)	Return an asynchronous consultation or tool result when supported.
checkpoint() / restore()	Persist and restore library state; record provider-state fidelity explicitly.
close()	Flush events, close transport, and return a final session summary.

Capability negotiation

Capability	Parlor/Gemma	Nova 2 Sonic	Manager response
Native speech-to-speech	No - cascaded local pipeline	Yes	Select prompt/audio strategy
Barge-in	Implemented by browser/runtime cancellation	Native event flow	Require normalized ResponseInterrupted
Asynchronous tools	Library orchestrates	Provider supports async tool flow	Choose provider-native or external dispatch
Session renewal	Runtime-controlled	Connection continuation required	Checkpoint before transition
Local/offline execution	Yes	No	Surface privacy/connectivity status
Text injection	Library-defined	Supported cross-modal input	Normalize as TextInputSubmitted

Do not design to the least capable provider.

Expose capabilities and adapt behavior. A lowest-common-denominator interface would erase Nova's native asynchronous tools and Parlor's local privacy advantage while still failing to make their session semantics identical.

Event model and session state

Append-only evidence plus compact mutable working state

Category	Initial event payloads
Transport	SessionStarted, SessionRenewing, SessionResumed, DeviceChanged, SessionClosed, ProviderError
Audio	AudioInputStarted, AudioInputStopped, PlaybackStarted, PlaybackStopped
Dialogue	TranscriptDelta, TranscriptFinalized, ResponseStarted, ResponseInterrupted, ResponseCompleted
Architecture	ConsultationRequested, InterventionReady, InterventionDelivered, DeepDiveRequested
Knowledge	ThreadOpened, ThreadParked, DecisionRecorded, ConstraintRecorded, QuestionOpened, QuestionResolved
Artifacts	ArtifactProposed, PatchReviewed, PatchApproved, PatchRejected, ArtifactApplied
Recovery	CheckpointCreated, CheckpointRestored, ReplayCompleted

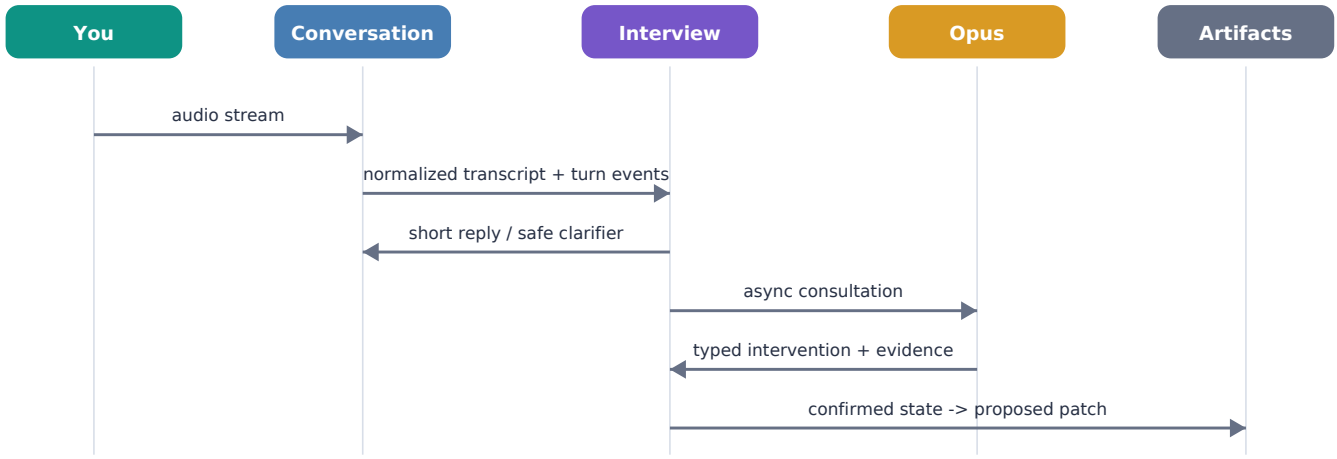
Envelope invariants

- session_id, monotonic sequence, UTC timestamp, source, correlation_id, causation_id, payload, and optional provider provenance
- events are immutable after append; corrections are new events
- interrupted and completed responses are distinct; a partial response is never silently upgraded to complete
- confirmed user statements remain distinguishable from model inference
- raw transcript is supporting evidence; compact thread state is the working context

Working state	Contents
Session summary	Problem, intended outcome, current scope, and latest synthesis
Thread index	Active, parked, merged, resolved, and superseded threads
Decision register	Decision, rationale, alternatives, confidence, confirming event
Constraint register	Source, affected components, hard/soft status, validation need
Open questions	Owner, blocking status, next experiment, resolution event

Runtime sequence and Opus intervention protocol

Conversation continues while the senior peer reasons in the background



Field	Purpose
urgency	background, next_boundary, or interrupt
kind	question, challenge, correction, finding, deep_dive_offer, or synthesis
speakable_text	Opus-authored wording suitable for near-verbatim delivery
supporting_detail	Technical reasoning retained for terminal review and artifacts
affected_threads	Stable thread identifiers, not labels inferred at delivery time
evidence	Repository facts, transcript event references, tool results, and confidence
expires_at / supersedes	Prevents stale interventions from re-entering a changed discussion

Gemma controls timing; Opus controls substance.

The facilitator may acknowledge, ask a safe clarifier, or choose the next conversational boundary. It should not compress away caveats or independently rewrite a material architectural correction.

CLI, configuration, and compact audio surface

The terminal is the control plane; the tiny window is the audio appliance

```
idea-architect providers list
idea-architect devices list

idea-architect start --provider parlor-gemma --architect bedrock-claude --repo .
idea-architect start --provider nova-sonic --architect bedrock-claude --repo .

idea-architect pause
idea-architect threads
idea-architect park
idea-architect resume
idea-architect review
idea-architect export  --format repo-diff
```

Configuration shape

Section	Key decisions
audio	Prefer Samsung input + output; fall back to Mac input + Samsung output, then Mac input + speakers on actual stream failure.
conversation	Provider name, provider options, session renewal policy, interruption threshold, compact-window launch mode.
architect	Provider, model identifier, consultation policy, max concurrency, timeout, and intervention delivery policy.
repository	Root, allowed read paths, ignored/generated/secret patterns, staging directory, apply mode disabled by default.
retention	Transcript/event/checkpoint locations, redaction rules, and per-session cleanup policy.
observability	Local structured logs and OTEL hooks without raw audio by default.

Compact window

Always visible	Expandable
Listening / thinking / speaking / paused	Input and output device selection
Session timer and latest transcript line	Provider and architect status
Mute, pause, and end	Latency, renewal, and error detail

Implementation roadmap

Immediate proof, near-term usefulness, then ecosystem integration

Phase	Deliverables	Exit gate
0 - Scaffold	Package, manager/ABC skeletons, event envelope, provider registry, CLI help, no-op providers	Core installs and tests without optional extras
1 - Parlor baseline	Pinned fork adapter, tiny app window, device routing, barge-in, transcript/events, checkpoint	Three stable 60-minute sessions
2 - Nova provider	Direct Bedrock bidirectional adapter first; renewal, async tools, transcript normalization	Same CLI/session tests pass for both providers
3 - Opus architect	Bedrock Claude provider, intervention schema, background/next-boundary/interrupt policies	Useful interventions exceed disruptive ones by a clear margin
4 - Interview state	Technical-peer policy, threads, confirmation, decisions, constraints, open questions	Session can recover and accurately summarize a 90-minute design
5 - Repo artifacts	Convention discovery, staging, Markdown providers, diff review, explicit apply	Claude Code can start from approved artifacts without transcript access
6 - Integration adapters	Framework integration type, typed agent, event bridge, optional Protocol Manager promotion	Standalone public API remains unchanged

Recommended build order inside Phase 2

Implement Nova 2 Sonic directly against Bedrock's bidirectional API before adding a Strands adapter. This gives the library a known provider contract and makes Strands an optional implementation choice rather than an accidental architectural dependency.

Near-term, not immediate

- Strands BidiAgent provider, once the direct Nova behavior establishes the contract
- AgentCore WebSocket/WebRTC hosting for shared or managed deployment
- automatic Claude Code handoff after an explicit review command
- semantic retrieval across prior sessions and repositories
- native Swift/Tauri menu-bar shell and global shortcuts

Testing, safety, and operational acceptance

Treat conversational behavior as a distributed real-time system

Layer	Required tests
Contract	Provider conformance suite; capability matrix; ordered events; close/idempotency; unsupported operations
Audio	Barge-in latency, false interruptions, turn-end quality, queue cancellation, device loss and fallback
Session	Renewal, crash recovery, checkpoint versioning, replay, out-of-order native events, duplicate delivery
Architect	Timeouts, stale responses, supersession, concurrent consultations, verbatim speakable delivery
Artifacts	Dirty worktree preservation, path allowlists, symlink escape, secret exclusion, deterministic patches
Long-running	60/90/120-minute soak tests with provider renewal, interruptions, pauses, and parked threads

Initial release metrics

Metric	Prototype target
Speech onset to playback stop	P95 under 300 ms
Unrecovered crash/audio deadlock	Zero in 90 minutes
Turn-end error rate	Below 5% on a labeled 100-turn set
State recovery	Active thread, parked threads, decisions, and transcript restored after forced restart
Artifact invention	Zero unmarked claims; confirmed and inferred items visibly distinct
Write authority	No repository mutation without explicit patch approval

Security boundary

The voice provider does not receive arbitrary repository contents. Repository context is selected by policy, redacted where appropriate, and attached to a specific architect consultation. Raw audio is not retained by default; transcripts and events have explicit retention settings.

Future framework integration

Integration type and typed agent are complementary, not competing forms

Framework form	Responsibility	Uses standalone library as
Integration type	Own session lifecycle, transport, authentication, event bridge, health, and external controls	A long-running communication channel
Typed interviewer agent	Ask, probe, challenge, summarize, and maintain structured interview state	A participant policy plugged into the channel
Typed architect agent	Provide Opus-level technical review and evidence-backed interventions	An ArchitectProvider implementation
Protocol Manager adapter	Promote confirmed decisions/evidence into versioned protocols and controlled effectors	An explicit downstream subscriber

Recommended eventual shape

- The integration owns connection and lifecycle; it does not become the interviewer.
- The typed agent owns behavior; it does not become the microphone or browser transport.
- The Opus shadow architect can be replaced with another typed agent without changing conversation providers.
- Protocol Manager subscribes to confirmed, typed events; it does not parse raw transcript text for authority decisions.
- TMNT or another domain agent may participate as an architect/tool peer without owning the session.

Integration readiness test

If bringing the prototype into the larger framework requires changing ConversationProvider, ConversationEvent, or checkpoint semantics, the standalone boundary was not finished. Framework integration should be adapter work.

Decisions, open questions, and first sprint

What is settled, what remains empirical, and what to build first

Settled decision	Rationale
Standalone Python library + CLI	Keeps the audio/product experiment independent of the larger framework.
Manager/factory/provider architecture	Two viable conversation backends already exist and differ materially in capabilities.
Parlor shell retained initially	It already solves much of the browser audio, turn, streaming, and interruption path.
Tiny app-mode window	Preserves browser audio advantages without adding a crowded browser work surface.
Opus is a separate ArchitectProvider	Voice orchestration and enterprise architecture reasoning are different responsibilities.
Reviewed artifacts only	The first prototype proves documentation fidelity, not autonomous effects.

Open questions

Open question	How to answer
Parlor fork or adapter-only integration?	Inspect current seams; prefer adapter until a required hook is missing.
Direct Nova SDK or Strands first?	Direct API first for contract discovery; compare effort after conformance tests exist.
Gemma model size/default?	Benchmark E2B/E4B/12B on response rhythm, transcript quality, and sustained thermals.
When should Opus interrupt?	Label real sessions and tune background/next-boundary/interrupt policy from observed value.
What repo context is safe and useful?	Start with explicit files plus workflow docs; expand only after provenance review.

First sprint

- Create the package skeleton, provider registry, capability model, event envelope, and no-op providers.
- Add a provider conformance test harness before implementing either real provider.
- Wrap stock Parlor behind ConversationProvider with no repo or Opus behavior.
- Launch the compact app-mode window and emit normalized transcript/interruption events.
- Run one 60-minute baseline session and record defects before starting Nova work.

References and design basis

Current-source check: 9 August 2026

[1] Parlor repository. Local Gemma 4 conversation, browser/FastAPI interaction, smart-turn, model setup, and project status.
https://github.com/fikrikarim/parlor

[2] Amazon Nova 2 Sonic getting started. Bedrock bidirectional speech-to-speech event flow and implementation baseline.
https://docs.aws.amazon.com/nova/latest/nova2-userguide/sonic-getting-started.html

[3] Nova 2 Sonic code examples. Full bidirectional, barge-in, tool use, resumption, and session-continuation examples.
https://docs.aws.amazon.com/nova/latest/nova2-userguide/sonic-code-examples.html

[4] Nova 2 Sonic asynchronous tools. Conversation can continue while tool calls remain pending.
https://docs.aws.amazon.com/nova/latest/nova2-userguide/sonic-async-tools.html

[5] Nova 2 Sonic cross-modal input. Text can be injected into an active streaming speech session.
https://docs.aws.amazon.com/nova/latest/nova2-userguide/sonic-cross-modal.html

[6] Strands voice and realtime quickstart. BidiAgent is available as an experimental bidirectional agent abstraction.
https://strandsagents.com/docs/user-guide/concepts/bidirectional-streaming/quickstart/

[7] Strands bidirectional events. Provider-neutral event and send/receive patterns for real-time conversations.
https://strandsagents.com/docs/user-guide/concepts/bidirectional-streaming/events/

[8] Strands session management. Persistence and restoration across bidirectional connection restarts.
https://strandsagents.com/docs/user-guide/concepts/bidirectional-streaming/session-management/

Scope assumptions

- Primary environment: native macOS on a MacBook Pro M5 Max with 128 GB unified memory and 4 TB storage.
- Primary audio route: Samsung Bluetooth earbud microphone and output, with Mac input/output fallbacks on failure.
- Primary workflow: terminal use alongside Claude Code from the active repository.
- Claude Opus is accessed through Amazon Bedrock; Anthropic's native voice interface is not assumed.
- The library may inspect and stage repository artifacts; repository mutation remains an explicit reviewed action.

Bottom line

The prototype is small enough to stand alone and strong enough to become infrastructure later. Its job is to prove an interruption-friendly design conversation, not to pre-build the whole agent ecosystem around it.